

capitolo 7 – Algoritmi per il calcolo numerico

1. Trovare gli zeri delle funzioni:
i metodi di bisezione e di Newton
2. Calcolare gli integrali definiti:
le formule del punto medio e del trapezio
3. Approssimare le derivate
con metodi a differenze finite
4. Risolvere equazioni differenziali ordinarie:
il metodo di Eulero esplicito

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Molti problemi si possono ridurre alla ricerca degli zeri di una funzione.

Identificare gli **zeri** di una funzione, cioè i valori di x per cui si ha $f(x) = 0$, significa trovare le ascisse dei punti in cui il grafico di $f(x)$ attraversa l'asse delle ascisse.

Anche per una «semplice» **equazione polinomiale**, del tipo:

$$a_0 + a_1x + a_2x^2 + \dots + a_nx^n = 0$$

il calcolo delle radici si può fare in modo analitico soltanto per $n \leq 4$.

Per $n > 4$ non esistono formule generali che consentano di determinare gli zeri del polinomio con un numero finito di operazioni.

A maggior ragione, ciò è vero quando si devono determinare le soluzioni di $f(x) = 0$ per una funzione f generica.

La **ricerca degli zeri** richiede l'uso di **tecniche numeriche di tipo iterativo**.

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** è un algoritmo che ha analogie con la **ricerca binaria**.

Consideriamo una **funzione** $f(x)$ che è **continua** nel dominio $[a,b]$, con $b > a$, e ha **segno opposto** ai due estremi: $f(a) \cdot f(b) < 0$.

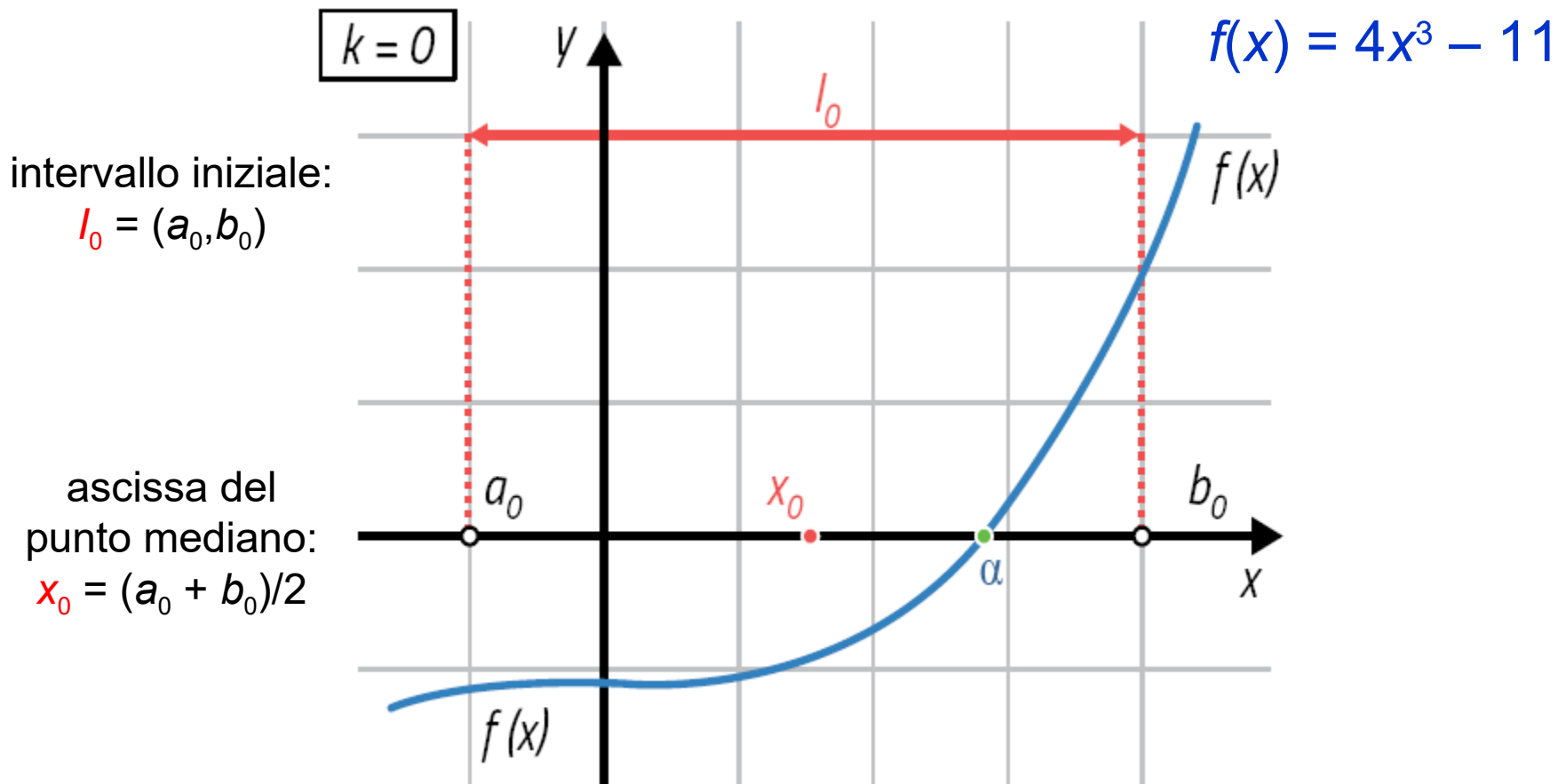
Certamente nell'intervallo (a,b) **esiste almeno uno zero** di f . Per semplicità supporremo che ci sia un unico zero, che indicheremo con α : $f(\alpha) = 0$.

Il **metodo di bisezione** consiste nel cercare α in intervalli di x sempre più piccoli, la cui ampiezza si dimezza a ogni iterazione.

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** è un algoritmo che ha analogie con la **ricerca binaria**.

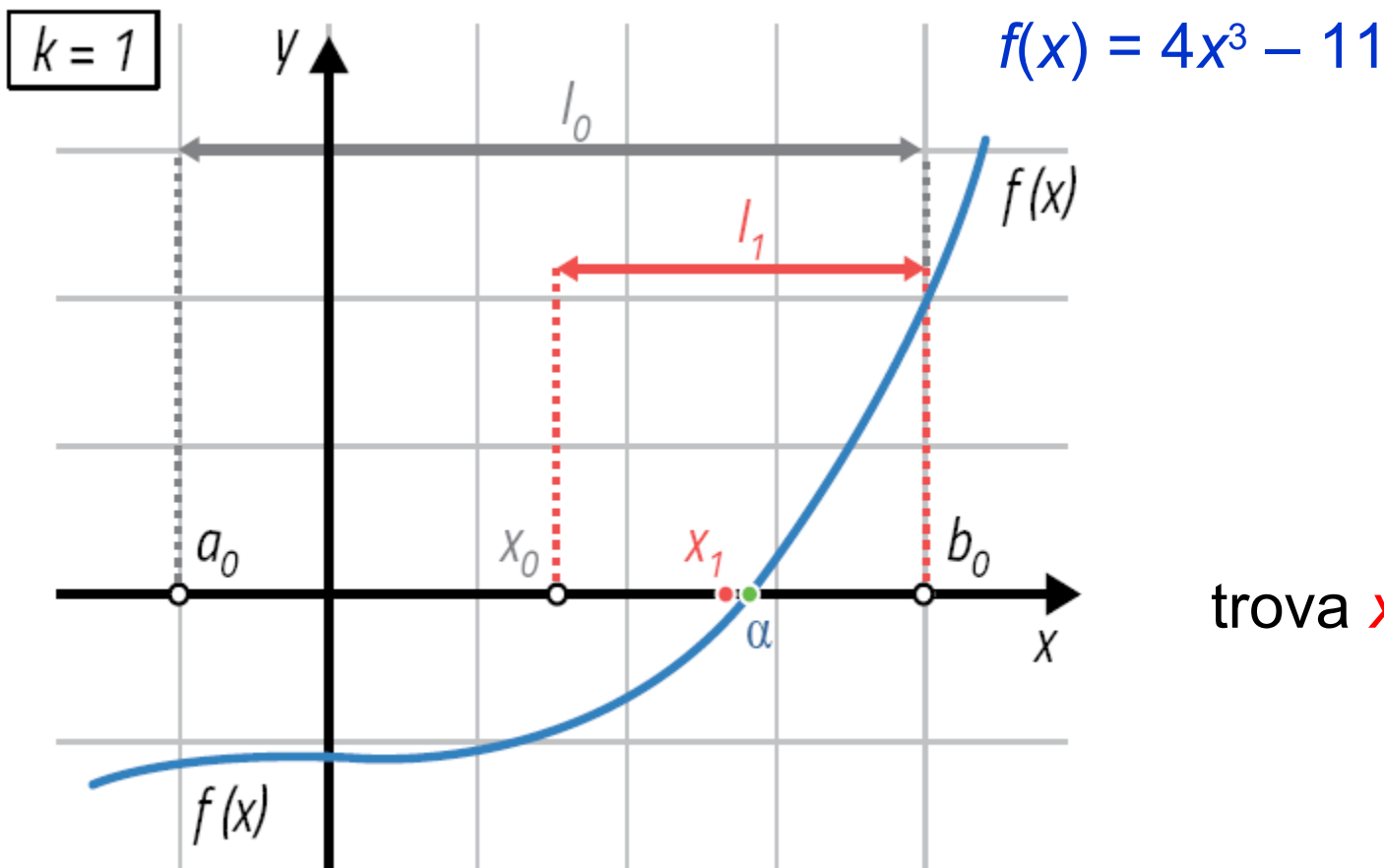


7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** è un algoritmo che ha analogie con la **ricerca binaria**.

prima
iterazione



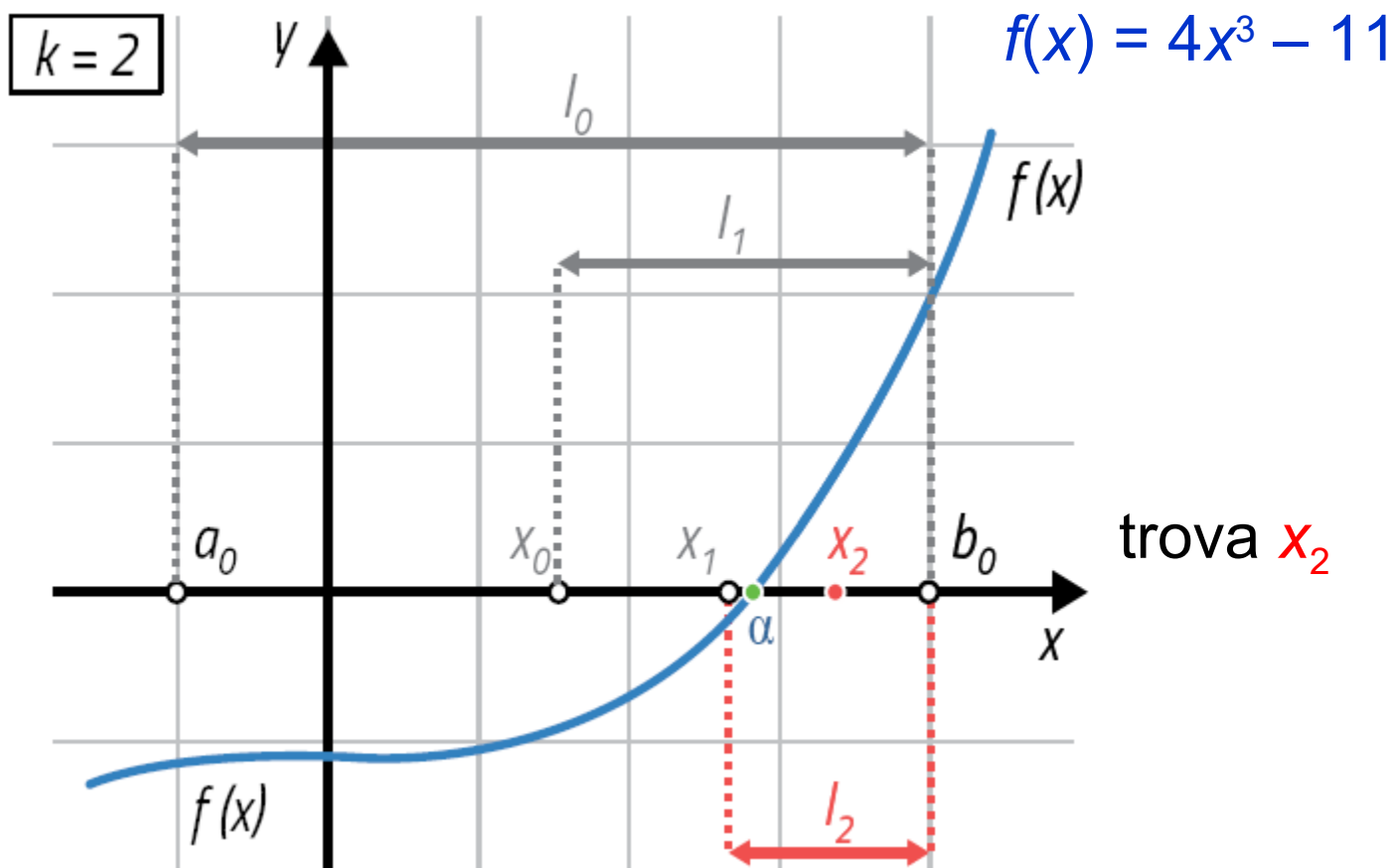
trova x_1

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** è un algoritmo che ha analogie con la **ricerca binaria**.

seconda
iterazione

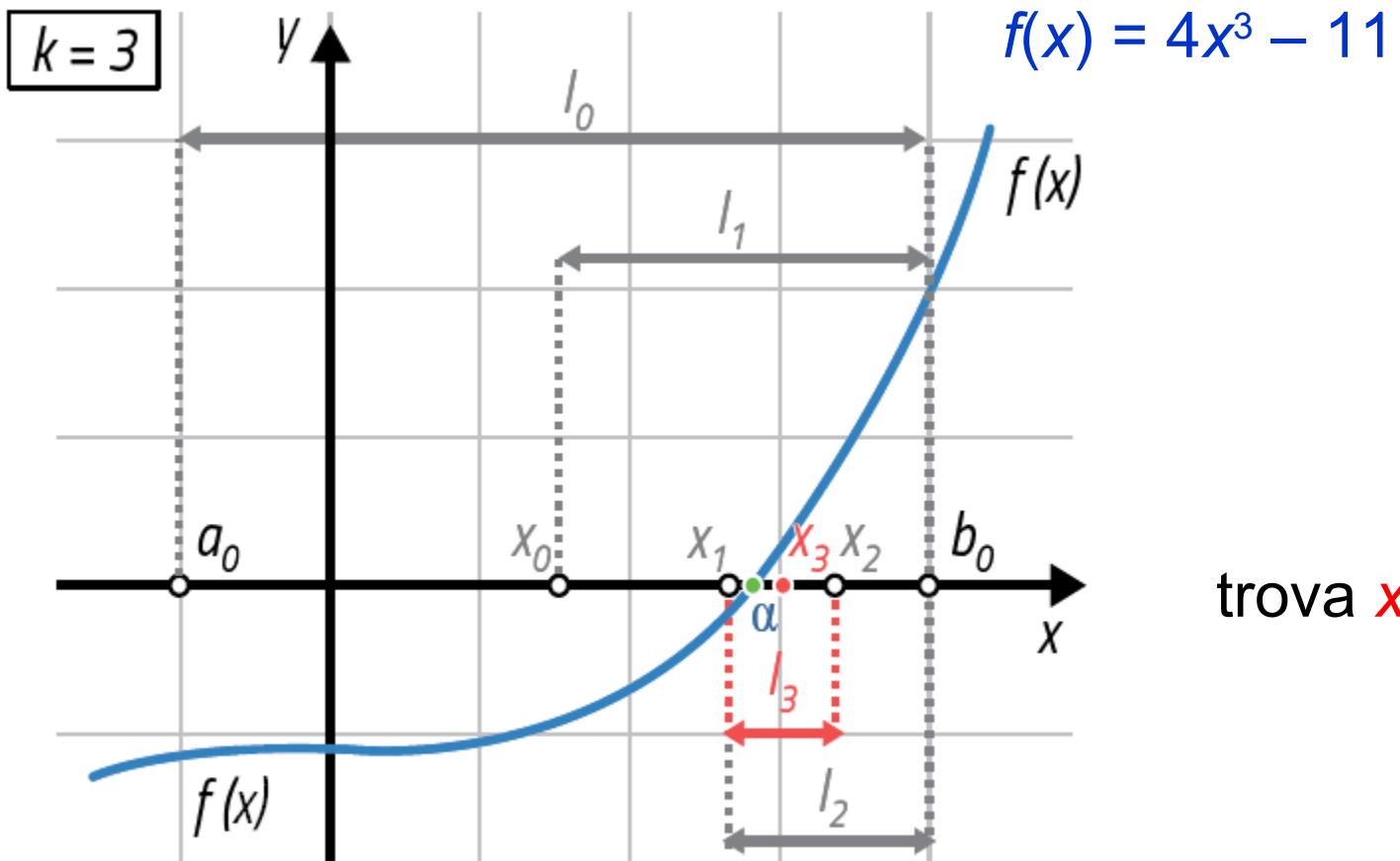


7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** è un algoritmo che ha analogie con la **ricerca binaria**.

terza
iterazione



trova x_3

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** garantisce la **convergenza** al risultato cercato.

Ciascuno degli intervalli I_k della successione contiene lo zero α cercato.

Inoltre **la convergenza** dell'algoritmo di bisezione **è assicurata**: infatti la successione dei valori $\{x_k\}$ converge sicuramente verso il valore α , giacché a ogni iterazione l'ampiezza dell'intervallo I_k , cioè $|I_k| = b_k - a_k$, si dimezza.

A ogni iterazione si trova dunque una soluzione approssimata che dista da α meno di *metà dell'ampiezza dell'intervallo considerato*.

L'**errore assoluto** e_k al passo k della procedura iterativa è quindi tale che:

$$|e_k| = |x_k - \alpha| < \frac{1}{2} |I_k| = \frac{1}{2^{k+1}} (b - a)$$

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di bisezione** garantisce la **convergenza** al risultato cercato.

$$|e_k| = |x_k - \alpha| < \frac{1}{2} |I_k| = \frac{1}{2^{k+1}} (b - a)$$

In generale però la soluzione α sarà un numero reale, **non rappresentabile** con la precisione finita del computer.

La condizione $x_k = \alpha$ dunque non potrà mai essere soddisfatta esattamente; questo **farebbe proseguire le iterazioni all'infinito**.

Per assicurare che l'algoritmo abbia un termine bisogna allora definire una **tolleranza ε** , cioè stabilire con quale precisione ci accontentiamo di ottenere il risultato: quando $|e_k| < \varepsilon$ interromperemo le iterazioni e x_k sarà il nostro risultato.

Dalla formula sopra consegue che **il numero delle iterazioni necessarie** con il metodo di bisezione è dato dal primo intero $k_{\min}$ che soddisfa la disuguaglianza:

$$k_{\min} > \log_2 \left(\frac{b - a}{\varepsilon} \right) - 1$$

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Un **programma** per trovare lo zero delle funzioni con il **metodo di bisezione**.

dati di input: gli estremi dell'intervallo e la tolleranza

```
funz.m x bisez.m x
1 function [soluz,res]=bisez(a,b,tol)
2 %-----
3 % Metodo di bisezione applicato alla funzione funz.m
4 % input: a,b (estremi dell'intervallo, con b>a), tol (tolleranza)
5 % output: soluz (zero cercato), res (residuo della funzione in xk)
6 %-----
7 disp('tolleranza richiesta: '); tol
8 kmin = ceil(log2(b-a)-log2(tol)-1);
9 disp('iterazioni necessarie: '); kmin
10 k=0; ak=a; bk=b; xk=(a+b)/2;
11 while k<=kmin
12     if funz(xk)*funz(ak)<0
13         bk=xk;
14     else
15         ak=xk;
16     endif
17     xk=(ak+bk)/2;
18     k=k+1;
19 endwhile
20 disp('ascissa trovata per lo zero: '); soluz=xk
21 disp('valore della funzione in xk: '); res=funz(xk)
22 endfunction
```

ceil() restituisce l'intero immediatamente superiore al valore dell'argomento

in Octave il ciclo di tipo **while** e la selezione **if...else** sono simili a quelli del C/C++ (senza graffe)

si applica a qualsiasi funzione continua **$f(x)$** definita nel file **funz.m**

```
funz.m x bisez.m x
1 function y=funz(x)
2     y = 4*x.^3-11;
3 endfunction
```

```
>> bisez(0.8,2.2,10^-4);
tolleranza richiesta:
tol = 0.00010000
iterazioni necessarie:
kmin = 13
ascissa trovata per lo zero:
soluz = 1.4010
valore della funzione in xk:
res = -0.00029643
```

output di Octave

Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

L'**algoritmo del metodo di Newton**, o **delle tangenti**, funziona come segue:

- si inizia scegliendo un'ascissa x_0 e **si traccia la tangente** in x_0 alla curva della funzione; **l'ascissa del punto in cui la tangente incrocia l'asse x** è il valore successivo x_1 ;
- poi **la procedura si ripete**: a ogni iterazione $k \geq 1$, si parte dall'ascissa x_k identificata al passo precedente e **si traccia la tangente alla funzione in x_k** : il punto in cui la tangente incrocia l'asse x determina il valore successivo x_{k+1} .

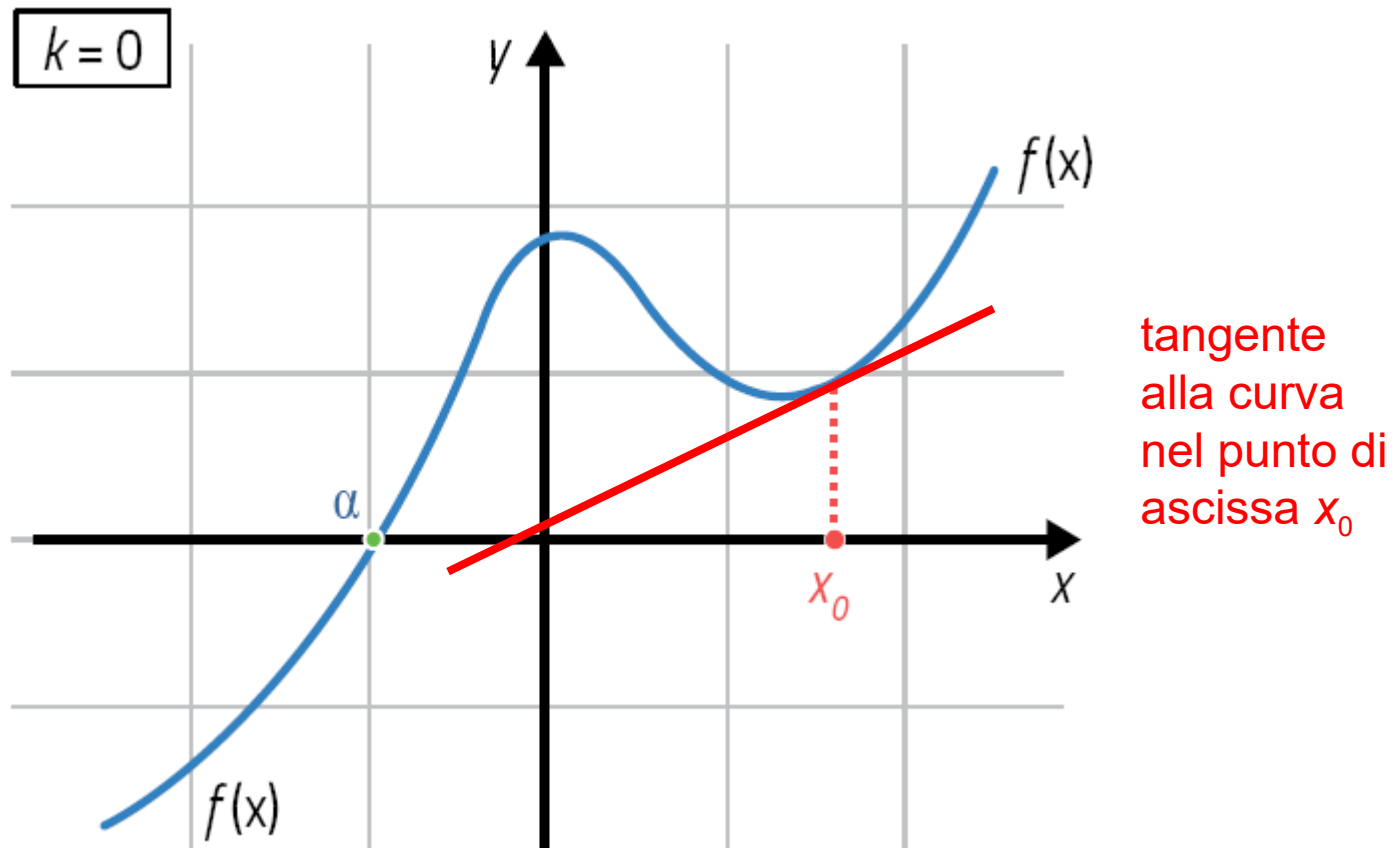
Lo zero di $f(x)$ dunque si identifica sostituendo localmente alla funzione la sua derivata $f'(x) = df(x)/dx$, coefficiente angolare della tangente al grafico di $f(x)$.

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

$$f(x) = 3x/2 + e^x + 15/(1+x^2) - 7$$



7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

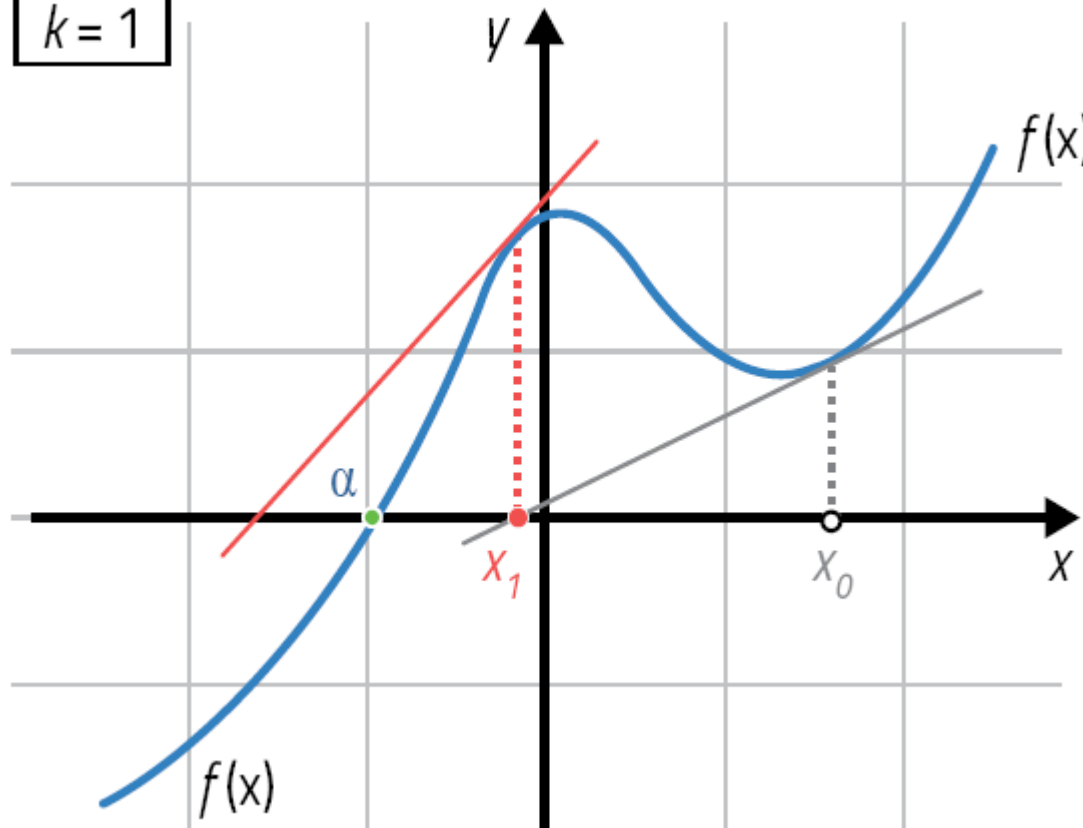
Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

$$f(x) = 3x/2 + e^x + 15/(1+x^2) - 7$$

prima
iterazione

$k = 1$

trova x_1



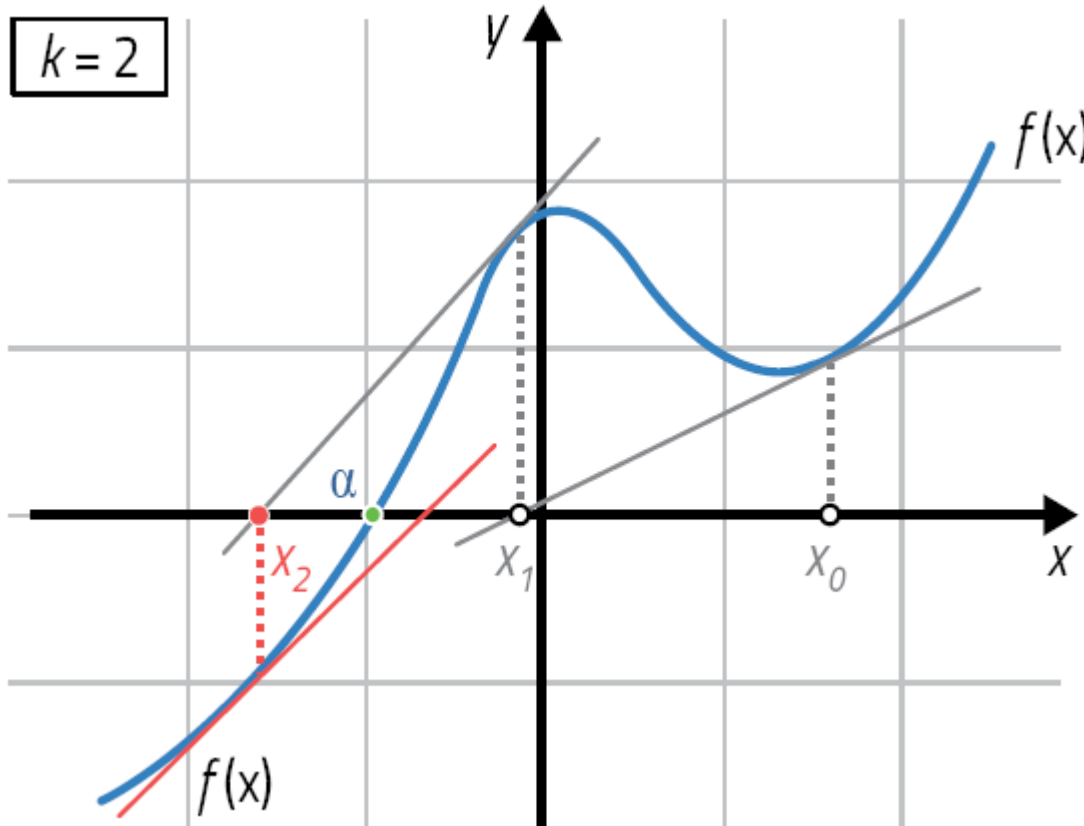
7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

$$f(x) = 3x/2 + e^x + 15/(1+x^2) - 7$$

seconda
iterazione



trova x_2

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

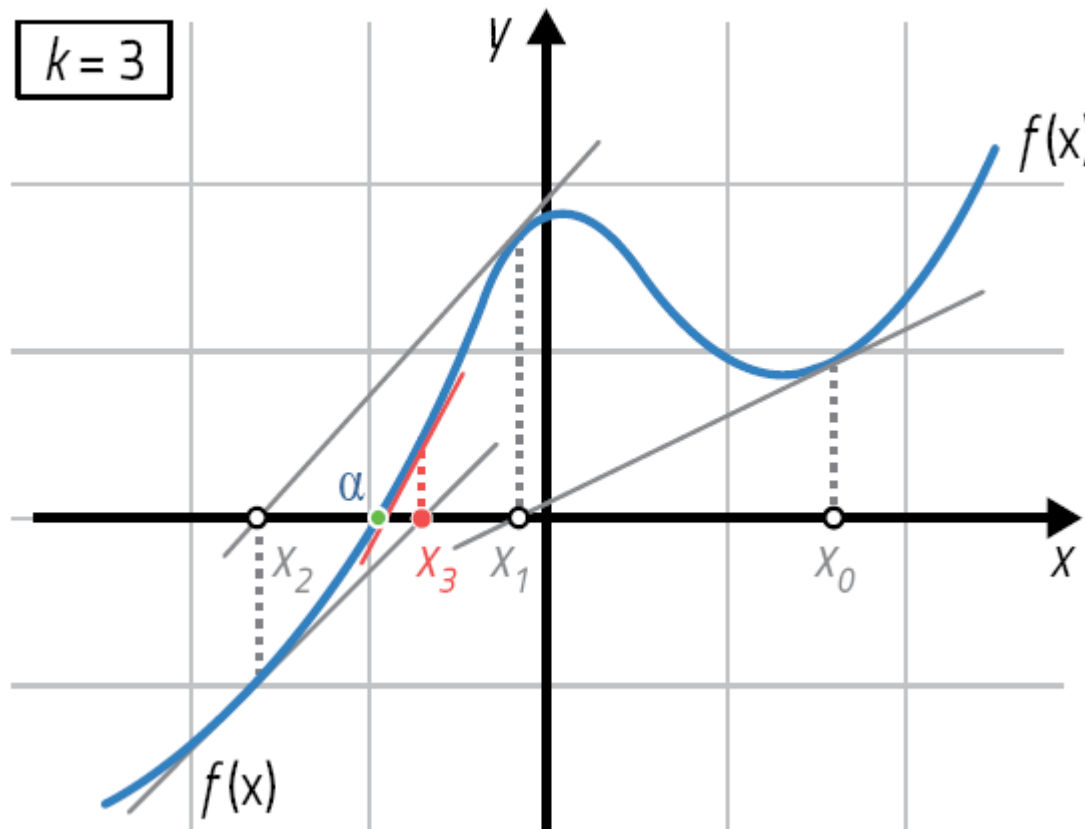
Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

$$f(x) = 3x/2 + e^x + 15/(1+x^2) - 7$$

terza
iterazione

$k = 3$

trova x_3



7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

L'equazione della retta tangente a f nel punto x_k è:

$$y(x) = f(x_k) + (x - x_k)f'(x_k)$$

e a ogni iterazione k del metodo si impone la condizione $y(x_{k+1}) = 0$.

Si ottiene così la formula del metodo di Newton:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Il metodo di Newton ***non converge necessariamente*** per qualsiasi valore dell'ascissa x_0 scelta come posizione iniziale.

Il vantaggio però è che, quando converge, lo fa molto più rapidamente rispetto al metodo di bisezione.

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Il **metodo di Newton**, o **delle tangenti**, è un'altra tecnica di calcolo numerico per **trovare gli zeri delle funzioni**.

Per imporre una **tolleranza** ε entro cui il risultato approssimato ottenuto sarà accettabile, si può procedere in due modi:

- **si stima l'errore come differenza tra le ascisse** trovate in iterazioni consecutive e si arresta il programma in corrispondenza del primo k per cui $|x_k - x_{k-1}| < \varepsilon$
- oppure **si impone la tolleranza sul residuo della funzione**, cioè sul valore trovato di $f(x_k)$, che sarebbe nullo se x_k fosse uno zero esatto; in tal caso si arresta il programma in corrispondenza del primo k per cui $|f(x_k)| < \varepsilon$.

Diversamente da quanto avviene con il metodo di bisezione, con quello di Newton **non si può prevedere quante iterazioni saranno necessarie** per raggiungere l'accuratezza prescritta.

Perciò è necessario specificare un **numero massimo di iterazioni** dopo cui far terminare l'algoritmo, che altrimenti potrebbe avvitarsi in un *loop* infinito.

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Un **programma** per trovare lo zero delle funzioni con il **metodo di Newton**.

input: ascissa iniziale, tolleranza e numero massimo di iterazioni

```
newton.m | funz.m | dfunz.m
1 function [soluz,res,niter]=newton(x0,tol,kmax)
2 %-----
3 % Metodo di Newton applicato a funz.m che ha derivata dfunz.m
4 % input: x0 (ascissa di partenza), tol (tolleranza),
5 %       kmax (limite al numero di iterazioni, evita loop infiniti)
6 % output: soluz (zero cercato), res (residuo della funzione in xk),
7 %        k (numero delle iterazioni fatte)
8 %-----
9 k = 0; diff = tol+1; xk = x0;
10 while abs(diff) >= tol && k < kmax
11     k = k + 1;
12     fxk = funz(xk);
13     dfxk = dfunz(xk);
14     diff = -fxk/dfxk;
15     xk = xk + diff;
16 endwhile
17 if (k==kmax && diff > tol)
18     disp('NB: raggiunto il numero massimo di iterazioni')
19     disp('senza aver raggiunto l''accuratezza richiesta')
20 endif
21 disp('ascissa trovata per lo zero: '); soluz=xk
22 disp('valore della funzione in xk: '); res = funz(xk)
23 disp('numero di iterazioni eseguite: '); niter=k
24 endfunction
```

la prima condizione del ciclo è basata sulla formula del metodo di Newton

qui servono due file .m, uno per la funzione $f(x)$ e uno per la derivata $f'(x)$

```
newton.m | funz.m | dfunz.m
1 function y=funz(x)
2     y=1.5*x+exp(x)+15/(1+x^2)-7;
3 endfunction
```

```
newton.m | funz.m | dfunz.m
1 function y=dfunz(x)
2     y=1.5+exp(x)-30*x/(1+x^2)^2;
3 endfunction
```

```
>> newton(1.6,10^-4,100);
ascissa trovata per lo zero:
soluz = -0.93433
valore della funzione in xk:
res =      8.8818e-16
numero di iterazioni eseguite:
niter = 7
```

output di Octave

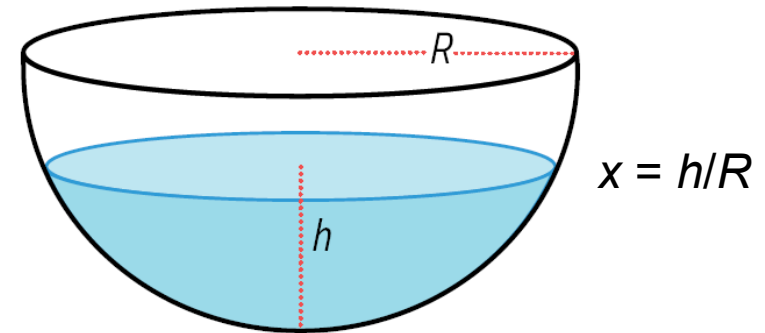
la selezione **if** avverte qualora si superi il numero massimo di iterazioni senza soddisfare la tolleranza

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Un'applicazione fisico-geometrica: il **problema di Archimede**.

A quale altezza h deve arrivare il liquido nel recipiente emisferico di raggio R perché il suo volume V_L sia esattamente **metà** del volume totale V del recipiente?



Dalla geometria sappiamo che per i due volumi valgono le formule:

$$V = \frac{2}{3}\pi R^3 \quad \text{e} \quad V_L = \frac{1}{3}\pi h^2 (3R - h) .$$

Perciò il problema richiede che si abbia:

$$\frac{V_L}{V} = \frac{h^2 (3R - h)}{2R^3} = \frac{1}{2} .$$

Se definiamo $x = h/R$, la condizione si può riscrivere nella forma:

$$x^3 - 3x^2 + 1 = 0$$

dove la nostra situazione fisica richiede che la variabile x sia compresa tra 0 e 1.

7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Un'applicazione fisico-geometrica: il **problema di Archimede**.

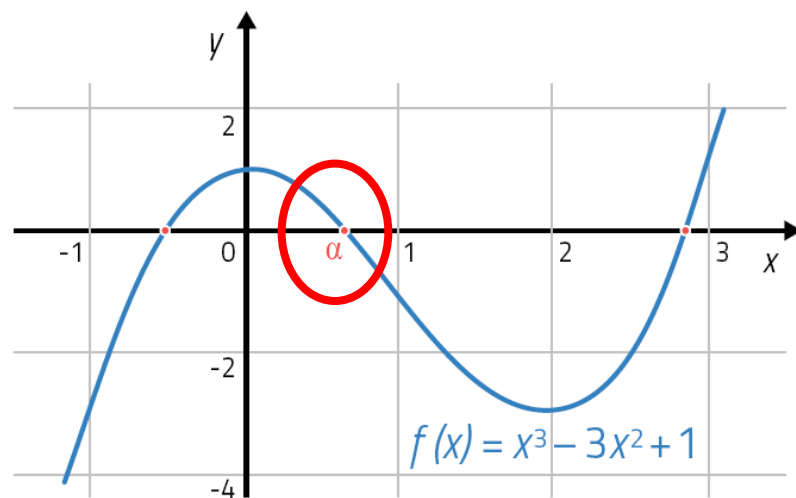
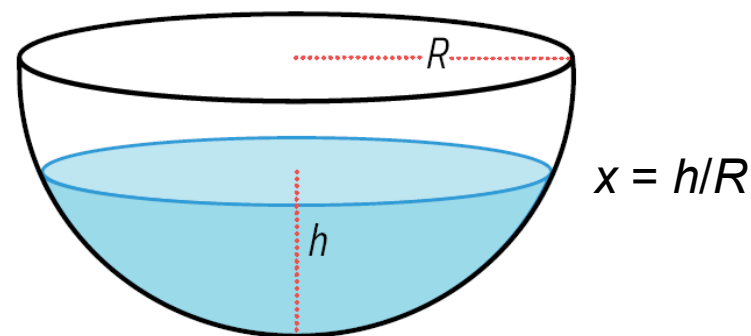
$$x^3 - 3x^2 + 1 = 0$$

Per un'equazione di terzo grado come questa esiste una **formula risolutiva** che dà un'espressione esplicita delle soluzioni.

Si tratta però di una formula complicata, che richiede di calcolare radici cubiche.

Cerchiamo allora una **soluzione numerica**, applicando i due metodi – di bisezione e di Newton – e confrontandone i risultati.

Il polinomio ha **tre zeri**, ma a noi interessa quello **centrale**: è l'unico compreso tra 0 e 1, come dev'essere la nostra variabile x .



7.1 Trovare zeri delle funzioni: i metodi di bisezione e di Newton

Indice

Un'applicazione fisico-geometrica: il **problema di Archimede**.

Eseguendo i due programmi **bisez** ($0, 1, 10^{-3}$) e **newton** ($0.5, 10^{-3}, 20$), con identico valore $\varepsilon = 10^{-3}$ per la tolleranza, si trovano i **risultati della tabella**.

**soluzione
analitica:
 $\alpha \cong 0.65270$**

k	metodo di bisezione	
	x_k	$f(x_k)$
0	0.50000	0.37500
1	0.75000	-0.26562
2	0.62500	0.072266
3	0.68750	-0.093018
4	0.65625	-0.0093689
5	0.64062	0.031712
6	0.64844	0.011236
7	0.65234	0.00094932
8	0.65430	-0.0042058
9	0.65332	-0.0016273
10	0.65283	-0.00033872

metodo di Newton	
x_k	$f(x_k)$
0.50000	0.37500
0.66667	-0.037037
0.65278	-0.00019558
0.65270	-0.0000000057248

l'algoritmo di Newton produce in soli 3 passi un'approssimazione migliore di quella che ha richiesto 10 passi con il metodo di bisezione

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Gli integrali definiti si devono spesso calcolare con metodi numerici.

Tra i problemi che si devono affrontare di frequente nell'analisi numerica c'è il calcolo approssimato di integrali definiti di funzioni $f(x)$:

$$I(f) = \int_a^b f(x) dx$$

Se $f(x)$ è una funzione limitata in $[a,b]$, il *teorema fondamentale del calcolo integrale* assicura l'esistenza di una funzione $F(x)$, detta **primitiva** di $f(x)$, tale che:

$$I(f) = F(b) - F(a) \quad .$$

Non tutte le funzioni, però, ammettono una primitiva esprimibile in termini di funzioni elementari. Inoltre, anche se è nota, la primitiva di $f(x)$ può essere complicata da valutare.

Ci sono poi tutti i casi, come l'acquisizione di dati sperimentali, in cui la funzione da integrare è disponibile soltanto in alcuni punti. **L'approssimazione numerica è allora la via obbligata per risolvere il problema.**

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Gli integrali definiti si devono spesso calcolare con metodi numerici.

I metodi numerici che stimano il valore di un integrale definito sono detti **formule di quadratura**. Qui esamineremo due semplici metodi che fanno parte di una famiglia più generale, quella delle cosiddette *formule di Newton-Cotes*.

Con questi metodi, quando si vuole stimare il valore dell'integrale definito tra a e b di una funzione $f(x)$, si adotta la seguente procedura:

- si suddivide l'intervallo (a,b) in N sottointervalli di uguale ampiezza $H = (b - a)/N$;
- si usa la proprietà di additività per riscrivere l'integrale come somma di N termini:

$$I(f) = \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x) dx$$

dove $x_i = a + iH$, con $i = 0, 1, \dots, N$;

- poi in ogni sottointervallo si sostituisce l'integrale di f con quello di un polinomio che in quel sottointervallo approssima f .

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Un **integrale definito** si può calcolare con la **formula del punto medio**.

La soluzione più semplice è quella basata sulla scelta del polinomio di grado 0, ossia costante, che interpola f nel *punto medio* $\bar{x}_i$ di ogni sottointervallo, cioè in:

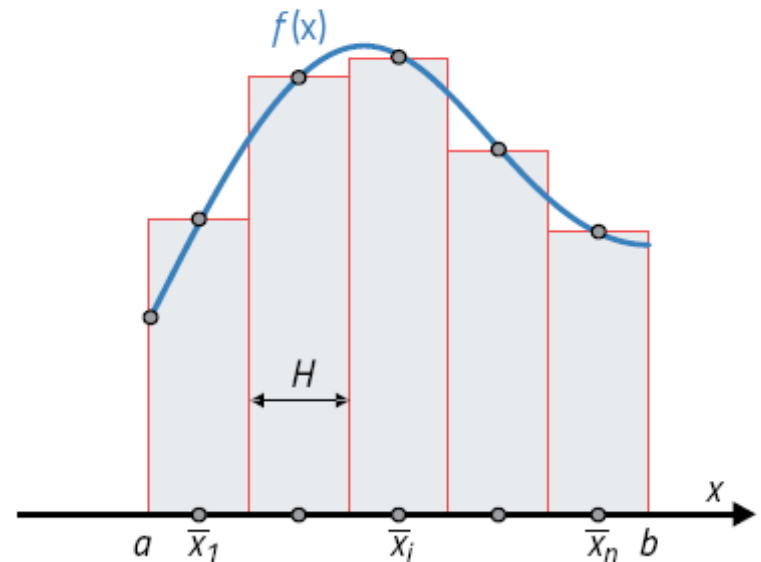
$$\bar{x}_i = \frac{x_{i-1} + x_i}{2}$$

In questo modo si ottiene la formula del punto medio che approssima l'integrale definito:

$$I_{pm}(f) = H \sum_{i=1}^N f(\bar{x}_i)$$

Il valore dell'integrale definito tra a e b di una funzione $f(x)$ è pari all'**area sottesa dal grafico** della funzione tra gli estremi a e b dell'intervallo.

La **formula del punto medio**, detta anche *formula dei rettangoli*, approssima l'area come mostrato dal grafico.



7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

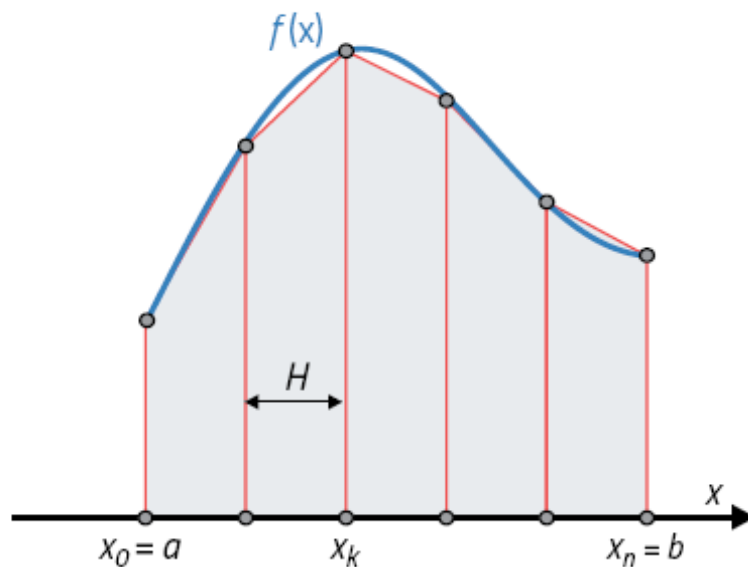
Indice

Un **integrale definito** si può calcolare anche con la **formula dei trapezi**.

Un'approssimazione alternativa si ottiene interpolando tra gli estremi dei sottointervalli con il polinomio di grado 1, ossia lineare.

In questo caso si ottiene la **formula dei trapezi**, che approssima l'integrale definito con la somma delle aree dei trapezi aventi come basi i valori $f(x_i)$ e $f(x_{i+1})$ e come altezza H :

$$I_t(f) = \frac{H}{2} \sum_{i=1}^N [f(x_i) + f(x_{i-1})] = \frac{H}{2} [f(a) + f(b)] + H \sum_{i=1}^{N-1} f(x_i)$$

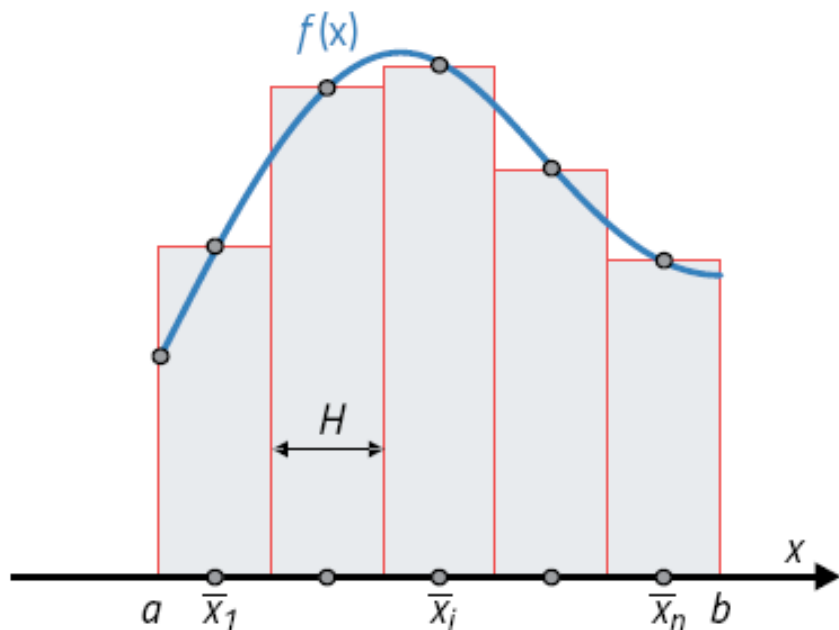


7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

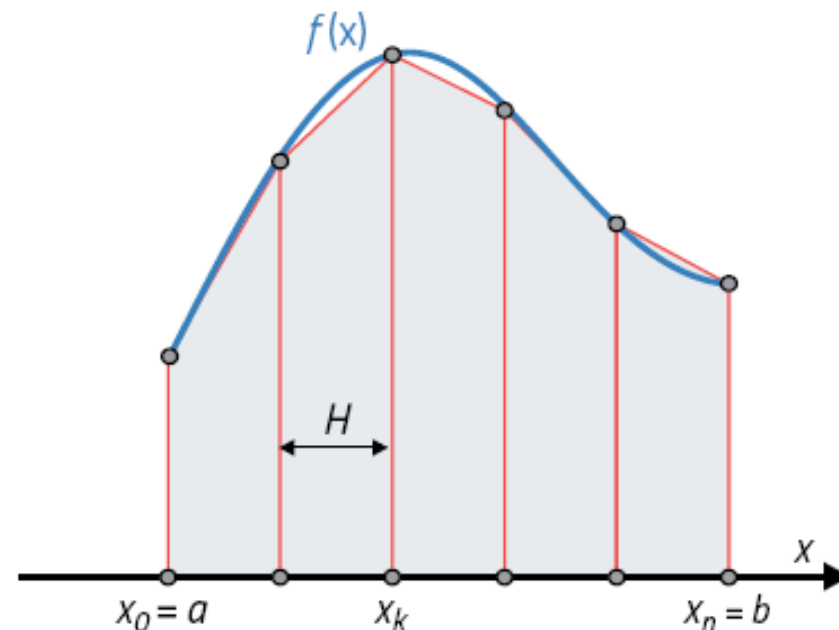
Indice

Un **confronto** tra il metodo del **punto medio** e il metodo dei **trapezi**.

metodo del punto medio



metodo dei trapezi



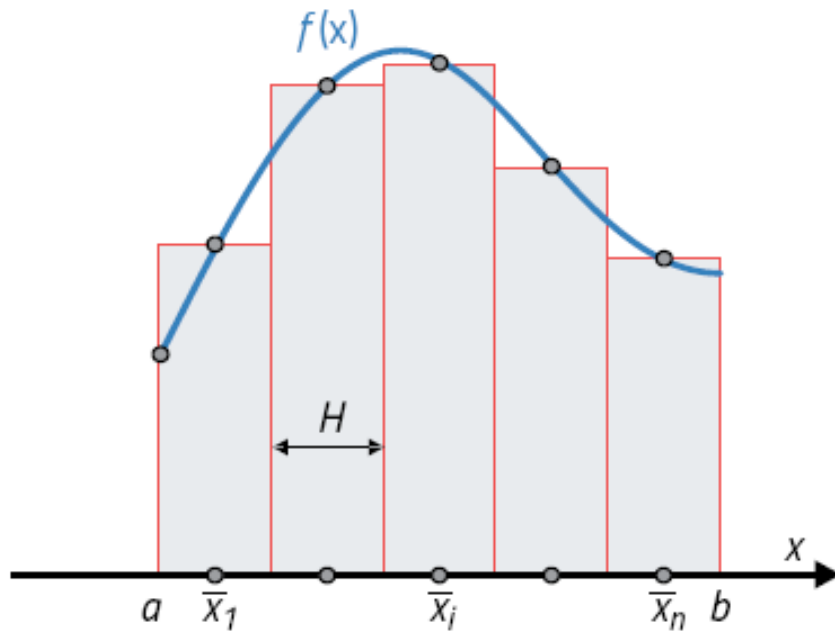
In questi disegni si ha $N = 5$. **Se N aumenta**, c'è da aspettarsi che con entrambi i metodi **l'approssimazione migliorerà**: l'involuppo superiore dei rettangoli e dei trapezi infatti assomiglierà sempre più alla curva della funzione.

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

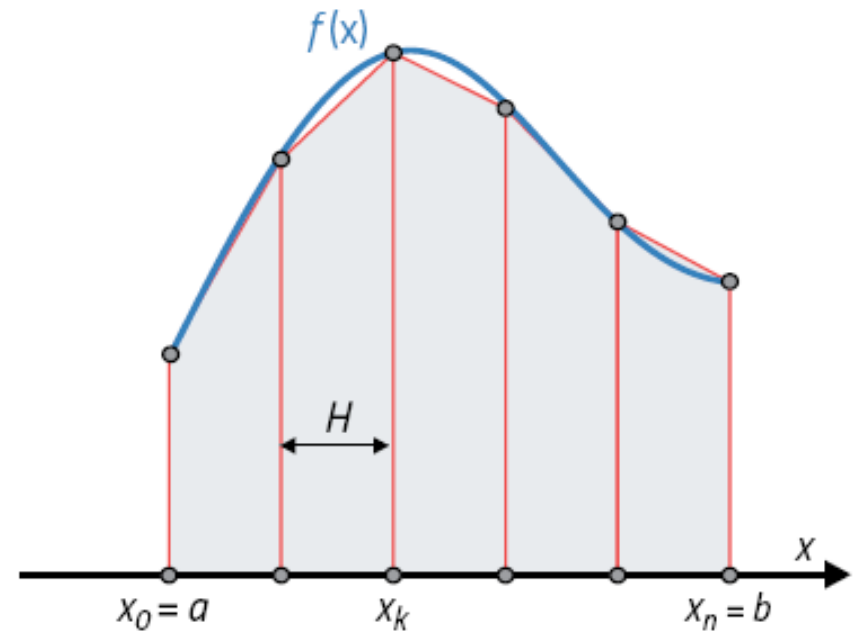
Indice

Un **confronto** tra il metodo del **punto medio** e il metodo dei **trapezi**.

metodo del punto medio



metodo dei trapezi



Ma quale tra i due metodi fornisce un'approssimazione migliore?

Per stabilirlo, la strategia migliore è codificarli e usare poi Octave per calcolare integrali in qualche caso esemplare, di cui sia nota la soluzione analitica esatta.

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Un **confronto** tra il metodo del **punto medio** e il metodo dei **trapezi**.

input: estremi dell'intervallo e numero di sottointervalli

```
1 function [Ipm, It] = integ_definito(a,b,N)
2 %-----
3 % Calcola l'integrale definito di funz.m tra a e b con due metodi
4 % e confronta i risultati con il valore esatto dato dalla primitiva
5 %-----
6 fprintf('\n \t a: %5.5f \t b: %5.5f \t N: %d', a,b,N); valore esatto
7 integr_esatto=primit(b)-primit(a); % valore calcolato dalla primitiva
8 fprintf('\n \t integrale esatto: %5.5f', integr_esatto);
9 H = (b-a)/N; % usiamo N suddivisioni di uguale ampiezza
10 % METODO 1: punto medio
11 Ipm = 0;
12 for i = 1:N
13     xmedioi = a + i*H - 0.5*H;
14     Ipm = Ipm + funz(xmedioi);
15 endfor
16 Ipm = H*Ipm;
17 relerr1=abs(integr_esatto-Ipm)/abs(integr_esatto); % errore relativo
18 fprintf('\n \t Ipm: %5.5f \t [rel.err.]: %2.2e', Ipm, relerr1);
19 % METODO 2: trapezi
20 It = 0.5*(funz(a)+funz(b));
21 for i = 1:N-1
22     xi = a + i*H;
23     It = It + funz(xi);
24 endfor
25 It = H*It;
26 relerr2=abs(integr_esatto-It)/abs(integr_esatto); % errore relativo
27 fprintf('\n \t It: %5.5f \t [rel.err.]: %2.2e \n', It, relerr2);
28 endfunction
```

metodo del punto medio

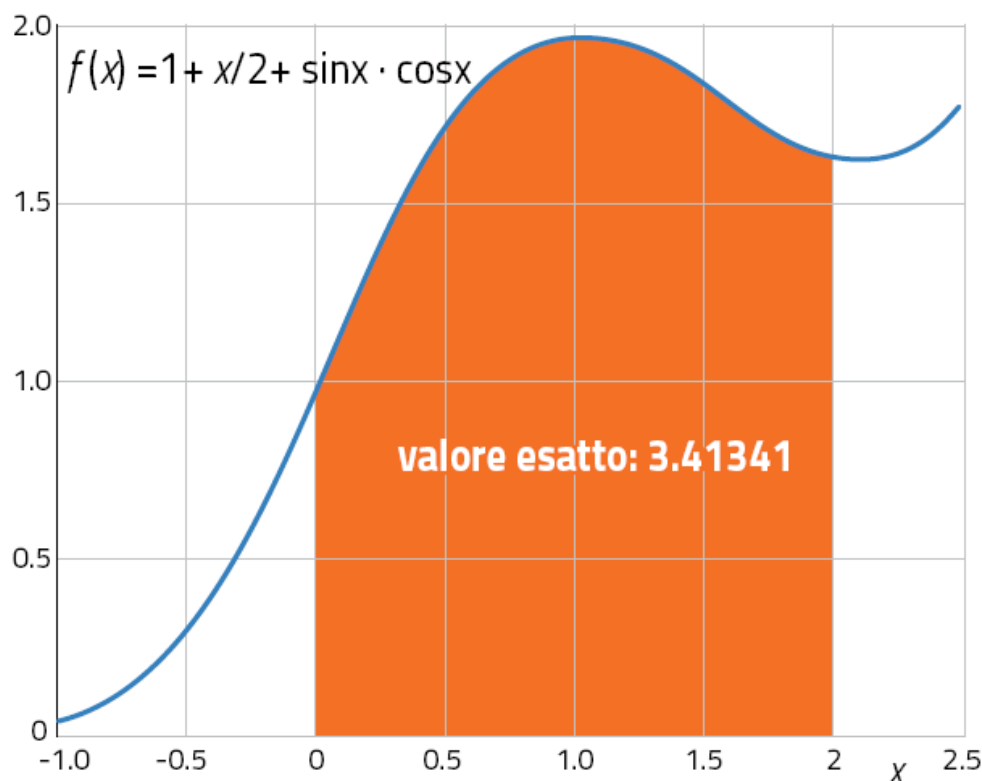
metodo dei trapezi

il programma
calcola l'integrale
di una funzione
funz che ha
primitiva **primit**

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Un **confronto** tra il metodo del **punto medio** e il metodo dei **trapezi**.



numero di intervalli	integrale approssimato	errore relativo
$N = 5$	$\begin{matrix} I_{pm} = 3.42464 \\ I_t = 3.39112 \end{matrix}$	$\begin{matrix} 3.29e-03 \\ 6.53e-03 \end{matrix}$
$N = 10$	$\begin{matrix} I_{pm} = 3.41618 \\ I_t = 3.40788 \end{matrix}$	$\begin{matrix} 8.11e-04 \\ 1.62e-03 \end{matrix}$
$N = 100$	$\begin{matrix} I_{pm} = 3.41344 \\ I_t = 3.41336 \end{matrix}$	$\begin{matrix} 8.07e-06 \\ 1.61e-05 \end{matrix}$
$N = 500$	$\begin{matrix} I_{pm} = 3.41341 \\ I_t = 3.41341 \end{matrix}$	$\begin{matrix} 3.23e-07 \\ 6.46e-07 \end{matrix}$

- l'ordine di grandezza dell'errore è lo stesso per i due metodi
- il metodo del punto medio è più accurato rispetto a quello dei trapezi
- al crescere di N , per entrambi i metodi l'errore diminuisce come $1/N^2$

7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

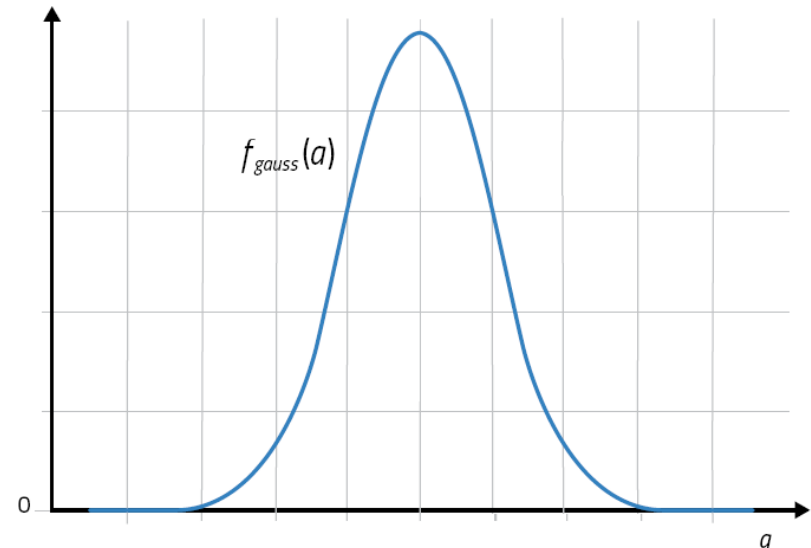
Un'applicazione statistica: **integrare la distribuzione normale.**

I fenomeni a **variazioni casuali intorno a un valore medio** si possono descrivere, almeno in prima approssimazione, mediante la **distribuzione di probabilità normale**, o **gaussiana**, descritta dalla funzione:

$$f_{gauss}(x) = \frac{M}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Se il numero M degli eventi considerati è sufficientemente grande, f_{gauss} rappresenta la **densità di probabilità** di ciascun valore reale x , dati il **valore medio** μ e la **varianza** σ della distribuzione.

Il **grafico** della distribuzione normale è la caratteristica **curva degli errori** a forma di campana, simmetrica rispetto a μ e con larghezza che aumenta al crescere di σ .



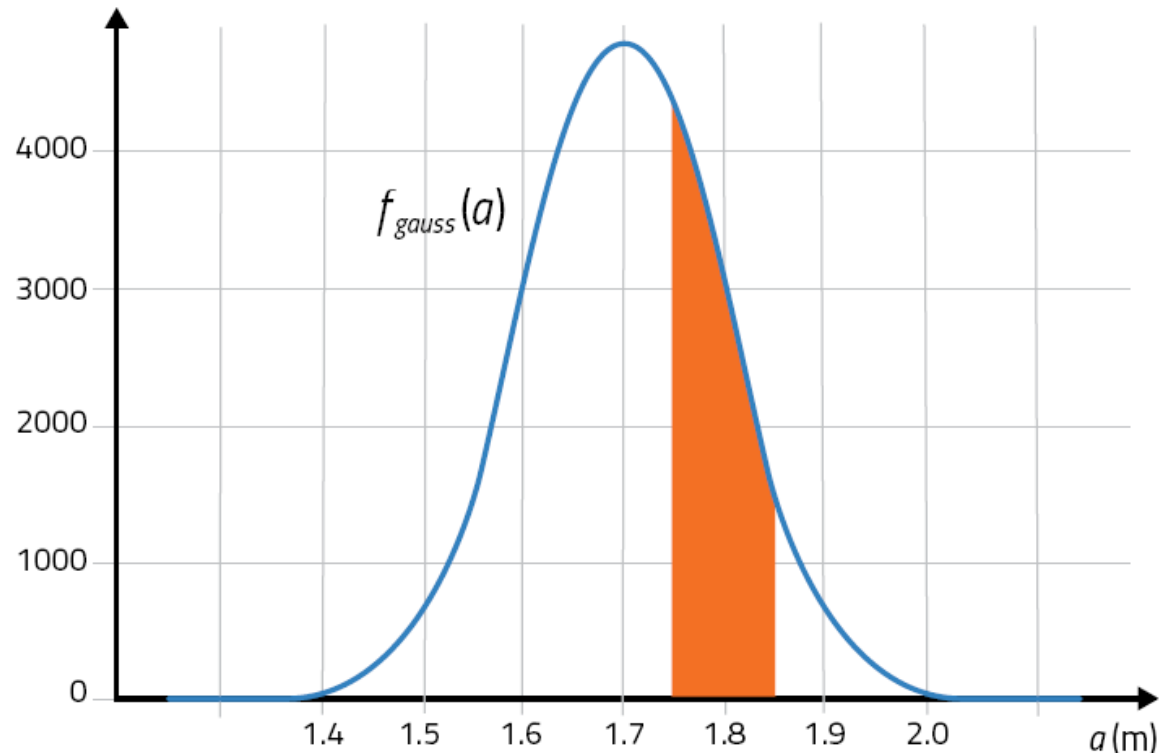
7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Un'applicazione statistica: **integrare la distribuzione normale.**

Questo grafico si riferisce a un gruppo di **$M = 1200$ persone** delle quali si è misurata l'altezza.

Il valor medio è $\mu = 1.70$ m e la varianza è $\sigma = 0.1$ m.



L'**integrale definito** corrispondente all'area colorata rappresenta il numero delle persone con altezza compresa tra 1.75 m e 1.85 m.

Quante sono queste persone?

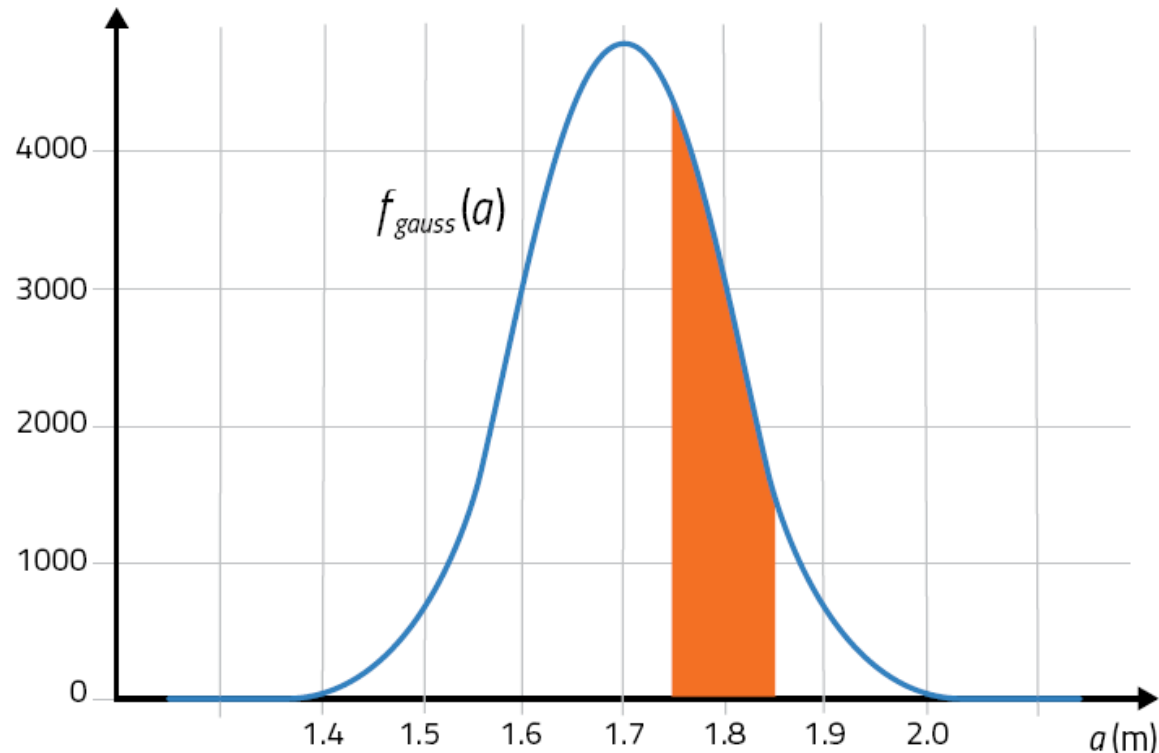
7.2 Calcolare integrali definiti: metodi di punto medio e trapezi

Indice

Un'applicazione statistica: **integrare la distribuzione normale.**

Questo grafico si riferisce a un gruppo di **$M = 1200$ persone** delle quali si è misurata l'altezza.

Il valor medio è $\mu = 1.70$ m e la varianza è $\sigma = 0.1$ m.



Eseguendo il programma Octave `integ_definito(1.75,1.85,100)`, si trova con entrambi i metodi – rettangoli e trapezi – lo stesso risultato: **290.076**.

Dunque **le persone con altezza compresa tra 1.75 e 1.85 m** risultano essere $290/1200 = 24\%$, vale a dire poco meno di **un quarto del totale**.

7.3 Approssimare le derivate con metodi a differenze finite

Indice

Per il calcolo numerico delle derivate si usano i metodi a differenze finite.

Se una funzione $f(x)$ è continua e derivabile con continuità in $[a,b]$, come si può approssimare la sua derivata $f'(\hat{x})$ in un generico punto $\hat{x}$ dell'intervallo (a,b) ?

Come sappiamo, la definizione della derivata in $\hat{x}$ prevede un passaggio al limite:

$$f'(\hat{x}) = \lim_{h \rightarrow 0} \frac{f(\hat{x} + h) - f(\hat{x})}{h}$$

ma questa prescrizione non è implementabile con un algoritmo, che non deve richiedere un numero infinito di passi.

Nel calcolo numerico si usano allora **differenze finite**, scegliendo un opportuno **valore discreto per h** e sostituendo il limite con un **rapporto incrementale**.

Esistono **tre diverse possibili approssimazioni** di questo tipo:

- differenze finite **in avanti**
- differenze finite **all'indietro**
- differenze finite **centrali**

7.3 Approssimare le derivate con metodi a differenze finite

Indice

Per il calcolo numerico delle **derivate** si usano i **metodi a differenze finite**.

differenze finite
in avanti

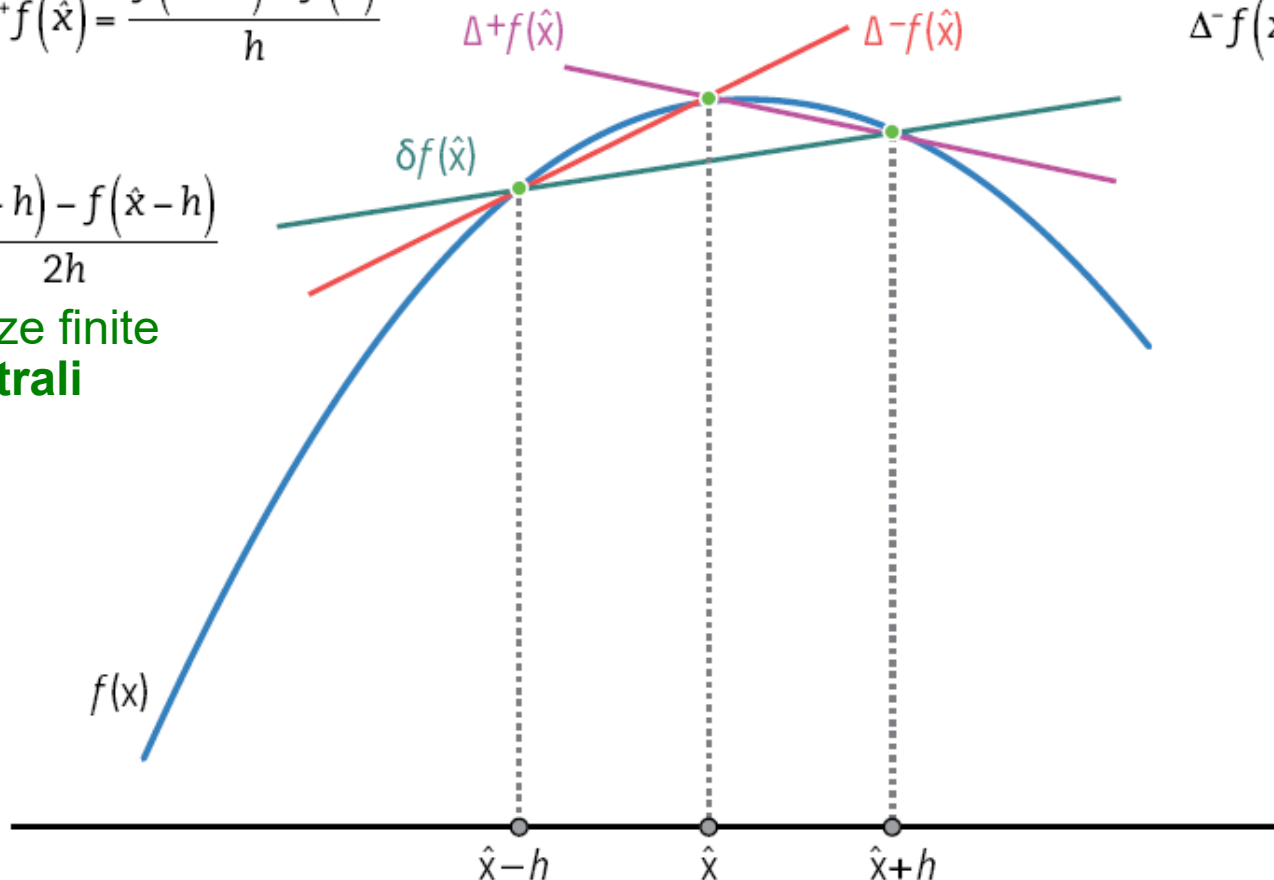
$$\Delta^+ f(\hat{x}) = \frac{f(\hat{x}+h) - f(\hat{x})}{h}$$

differenze finite
all'indietro

$$\Delta^- f(\hat{x}) = \frac{f(\hat{x}) - f(\hat{x}-h)}{h}$$

$$\delta f(\hat{x}) = \frac{f(\hat{x}+h) - f(\hat{x}-h)}{2h}$$

differenze finite
centrali



7.3 Approssimare le derivate con metodi a differenze finite

Indice

Per il calcolo numerico delle **derivate** si usano i **metodi a differenze finite**.

- › differenze finite in avanti: $\Delta^+ f(\hat{x}) = \frac{f(\hat{x} + h) - f(\hat{x})}{h}$
- › differenze finite all'indietro: $\Delta^- f(\hat{x}) = \frac{f(\hat{x}) - f(\hat{x} - h)}{h}$
- › differenze finite centrali: $\delta f(\hat{x}) = \frac{f(\hat{x} + h) - f(\hat{x} - h)}{2h}$

Dallo sviluppo di $f(x)$ in serie di Taylor si può dedurre che $\Delta^+ f(\hat{x})$ e $\Delta^- f(x)$ sono approssimazioni *del primo ordine* della derivata $f'(\hat{x})$: ciò significa che, quando h diminuisce, l'errore assoluto sulla derivata tende a zero come h .

Le differenze finite centrali danno invece un'approssimazione migliore, perché è *del secondo ordine* e quindi più accurata: l'errore assoluto sulla derivata tende a zero come h^2 .

7.3 Approssimare le derivate con metodi a differenze finite

Indice

Un **confronto** quantitativo in Octave fra i tre **metodi a differenze finite**.

calcola la derivata numerica di **funz** nel punto **x0** con i tre metodi e la confronta con il valore della derivata analitica **dfunz (x0)**

```
diff_finite.m
1 function [Deltap,Deltam,delta] = diff_finite(x0,h)
2 %-----
3 % Calcola la derivata della funzione funz.m in x0 con tre metodi.
4 %-----
5 fprintf('\n \t x0: %5.5f \t h: %5.5f',x0,h);
6 deriv_esatta=dfunz(x0); % valore esatto della derivata
7 fprintf('\n \t derivata esatta: %5.5f',deriv_esatta);
8 % METODO 1: differenze finite in avanti
9 Deltap=(funz(x0+h)-funz(x0))/h;
10 relerr1=abs(deriv_esatta-Deltap)/abs(deriv_esatta); % errore relativo
11 fprintf('\n \t Deltap: %5.5f \t [rel.err.]: %2.2e',Deltap,relerr1);
12 % METODO 2: differenze finite all'indietro
13 Deltam=(funz(x0)-funz(x0-h))/h;
14 relerr2=abs(deriv_esatta-Deltam)/abs(deriv_esatta); % errore relativo
15 fprintf('\n \t Deltam: %5.5f \t [rel.err.]: %2.2e',Deltam,relerr2);
16 % METODO 3: differenze finite centrali
17 delta=(funz(x0+h)-funz(x0-h))/(2*h);
18 relerr3=abs(deriv_esatta-delta)/abs(deriv_esatta); % errore relativo
19 fprintf('\n \t delta: %5.5f \t [rel.err.]: %2.2e \n',delta,relerr3);
20 endfunction
```

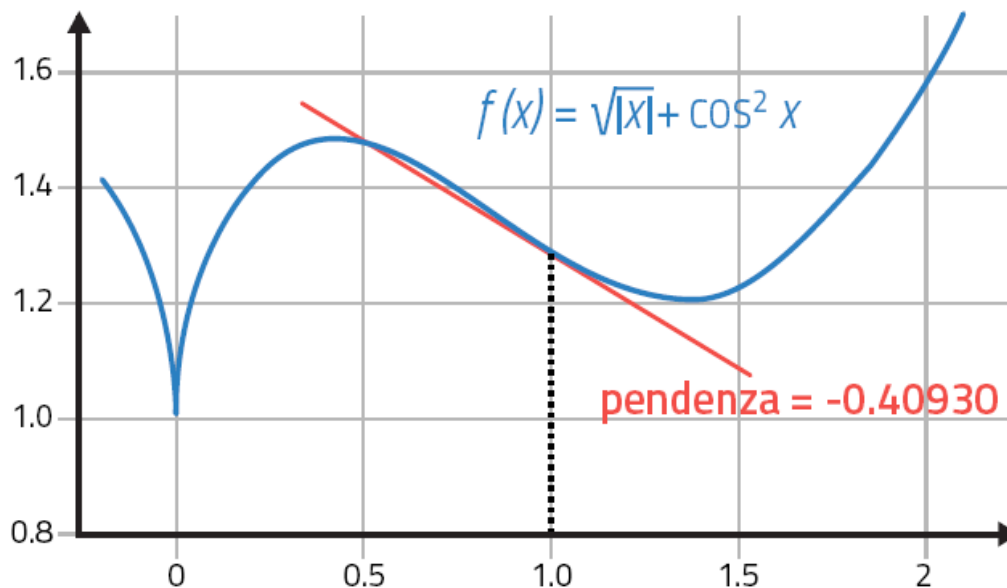
7.3 Approssimare le derivate con metodi a differenze finite

Indice

Un **confronto** quantitativo in Octave fra i tre **metodi a differenze finite**.

la **derivata esatta** in $x=1$ è $f'(1) = -0.40930$

i risultati per i tre metodi
al variare dell'incremento h



incremento	derivata approssimata	errore relativo
$h = 0.5$	$\Delta^+ f(\hat{x}) = -0.12436$ $\Delta^- f(\hat{x}) = -0.37066$ $\delta f(\hat{x}) = -0.24751$	$6.96e-01$ $9.44e-02$ $3.95e-01$
$h = 0.1$	$\Delta^+ f(\hat{x}) = -0.37368$ $\Delta^- f(\hat{x}) = -0.43156$ $\delta f(\hat{x}) = -0.40262$	$8.70e-02$ $5.44e-02$ $1.63e-02$
$h = 0.01$	$\Delta^+ f(\hat{x}) = -0.40632$ $\Delta^- f(\hat{x}) = -0.41214$ $\delta f(\hat{x}) = -0.40923$	$7.28e-03$ $6.95e-03$ $1.63e-04$
$h = 0.001$	$\Delta^+ f(\hat{x}) = -0.40901$ $\Delta^- f(\hat{x}) = -0.40959$ $\delta f(\hat{x}) = -0.40930$	$7.13e-04$ $7.10e-04$ $1.63e-06$

- l'**approssimazione migliore** si ottiene con le **differenze finite centrali**; con questo metodo infatti l'errore relativo tende a essere sensibilmente inferiore
- **al diminuire dell'incremento h** per tutti e tre i metodi **aumenta l'accuratezza** della stima ottenuta per la derivata

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un'**equazione differenziale** contiene derivate di una funzione incognita.

Un'**equazione differenziale ordinaria** è un'equazione contenente **derivate** della funzione incognita fatte **rispetto a un'unica variabile indipendente**.

Indicheremo questa variabile con **t** , iniziale di **tempo**; di solito infatti l'equazione differenziale è il modello matematico dell'evoluzione di un fenomeno nel tempo.

Esempio: la legge fondamentale della dinamica **$a = F/m$** , che regola il movimento degli oggetti, è un'equazione differenziale ordinaria.

Infatti l'accelerazione **a** è la derivata seconda della posizione rispetto al tempo e l'intensità della forza **F** in generale dipende dalla posizione e cambia nel tempo.

Qui considereremo il caso delle equazioni differenziali **del primo ordine**, in cui cioè appare **soltanto la derivata prima della funzione incognita**.

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero

Indice

Il **problema di Cauchy** consiste in un'**equazione differenziale** con un **valore iniziale** per la soluzione.

In generale un'**equazione differenziale ordinaria** ha infinite soluzioni diverse.

Per **identificare una particolare soluzione** bisogna **imporre una condizione** che prescriva il valore assunto dalla soluzione in un punto dell'intervallo di integrazione.

Si ha allora un **problema di Cauchy**, che consiste nel trovare una funzione $y(t)$ tale che, in un dato intervallo di tempo:

$$\begin{cases} y'(t) = f(t, y(t)) \\ y(t_0) = y_0 \end{cases}$$

dove f è una funzione data, che dipende dalla funzione $y(t)$ ed eventualmente dal tempo t anche in altri modi.

Il **tempo** t_0 è quello a cui è dato il **valore noto della funzione incognita**; di norma si tratta dell'**istante di tempo iniziale** $t_0 = 0$.

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero **Indice**

Il metodo di Eulero esplicito è la tecnica più semplice per risolvere il problema di Cauchy.

Per poter trattare il problema continuo mediante un algoritmo numerico discreto, si suddivide l'intervallo di tempo studiato $[t_0, T]$ in N sottointervalli, che hanno tutti ampiezza pari al passo di discretizzazione $h = (T - t_0)/N$ detto *time step*, cioè «gradino temporale».

Poi per ogni nodo $t_n = t_0 + nh$ (dove $n = 1, 2, \dots, N$) si cerca un valore y_n che approssimi quello della funzione cercata, $y(t_n)$. L'insieme dei valori $\{y_0, y_1, y_2, \dots, y_N\}$ formerà la soluzione numerica del problema di Cauchy.

A ogni nodo si usa la derivata rispetto al tempo $y'(t_n)$, che è data dal secondo membro dell'equazione differenziale, per far variare y e calcolare il suo valore approssimato nel nodo successivo, cioè y_{n+1} .

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero

Indice

Il metodo di Eulero esplicito è la tecnica più semplice per risolvere il problema di Cauchy.

Una tecnica classica è il metodo di Eulero in avanti, analogo a quello delle differenze finite in avanti con cui abbiamo approssimato le derivate.

La derivata temporale di y viene approssimata con un rapporto incrementale:

$$y'(t_n) \approx \frac{y_{n+1} - y_n}{h},$$

e da qui, usando l'equazione differenziale e scrivendo per brevità $f_n = f(t_n, y_n)$, si ottiene:

$$y_{n+1} = y_n + h f_n \quad \text{per} \quad n = 0, 1, \dots, N-1$$

Questa è la *formula del metodo di Eulero in avanti*, detto anche *esplicito* perché la soluzione numerica y_{n+1} dipende esclusivamente dal valore y_n già calcolato in precedenza.

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un **programma** in Octave che implementa il **metodo di Eulero esplicito**.

```
eulero.m ✖
1 function [y, yesatta] = eulero(t0,T,y0,N)
2 %-----
3 % Risolve l'ODE y'(t)=funz(t) con il metodo di Eulero "in avanti"
4 % per il caso-test in cui funz(t) e' la derivata della gaussiana
5 %-----
6 h = (T-t0)/N;           % passo di discretizzazione
7 t = t0:h:T;           % vettore dei tempi
8 mu = 1.0; sigma=1.00; y0 = exp(-mu^2/sigma^2);
9 y(1) = y0; yesatta(1)=y0;
10 for n=1:N
11     funz(n)=-2.*(t(n)-mu)./sigma^2.*y(n);
12     y(n+1) = y(n) + h*funz(n);
13     yesatta(n+1)=exp(-(t(n+1)-mu).^2/sigma^2);
14 endfor
15 figure; % il grafico confronta la soluzione numerica e quella esatta
16 plot(t, y, 'r--', 'Linewidth', 2, t, yesatta, 'g-', 'Linewidth', 2);
17 legend('approssimata', 'esatta');
18 title('metodo di Eulero con N=500');
19 endfunction
```

input: intervallo di tempo, valore iniziale, passo di discretizzazione

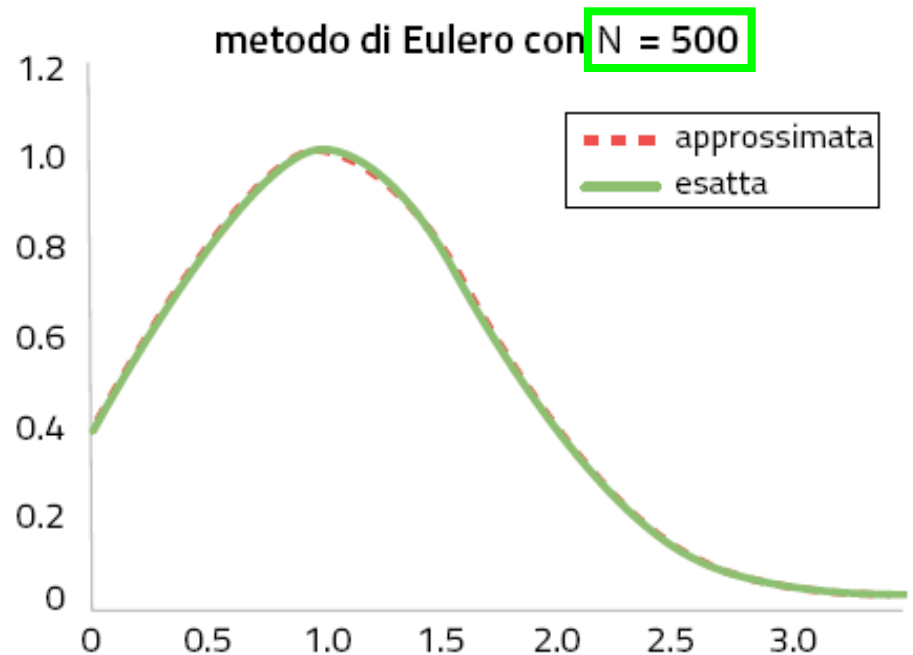
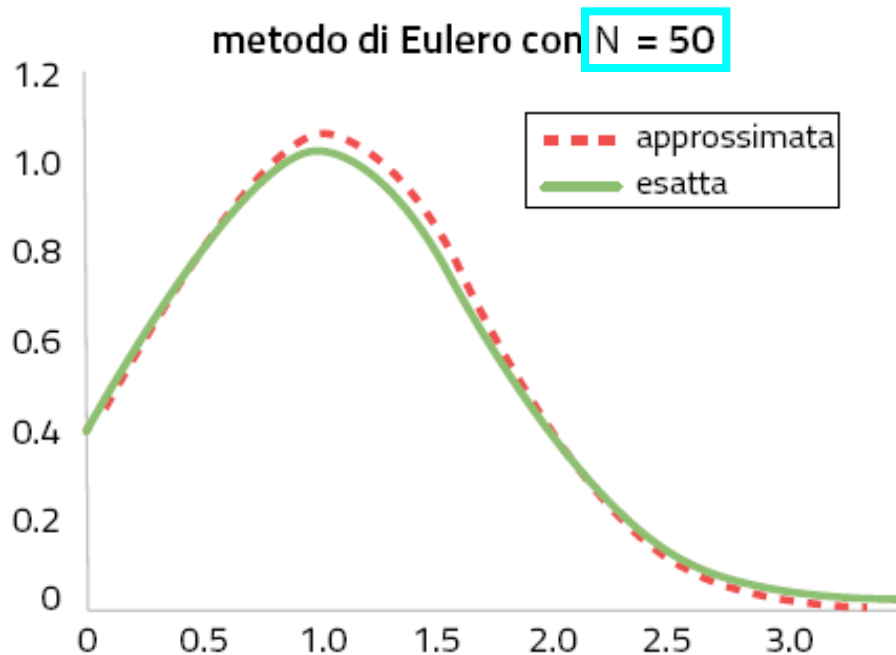
l'equazione differenziale
qui è $y'(t) = 2(t - 1)y(t)$

formula di Eulero

la primitiva $y(t)$ qui è la gaussiana con $M = \mu = \sigma = 1$

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un **programma** in Octave che implementa il **metodo di Eulero esplicito**.



qui il **passo h** usato per l'integrazione numerica è **ridotto** di un fattore 10:
l'approssimazione numerica migliora

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un'applicazione fisica: **simulazione** della **caduta libera di un paracadutista**.

Un paracadutista salta dall'aereo con **velocità verticale iniziale nulla**.

Mentre scende con un'accelerazione verticale diretta verso il basso, incontra una **resistenza** sempre maggiore da parte **dell'aria**.

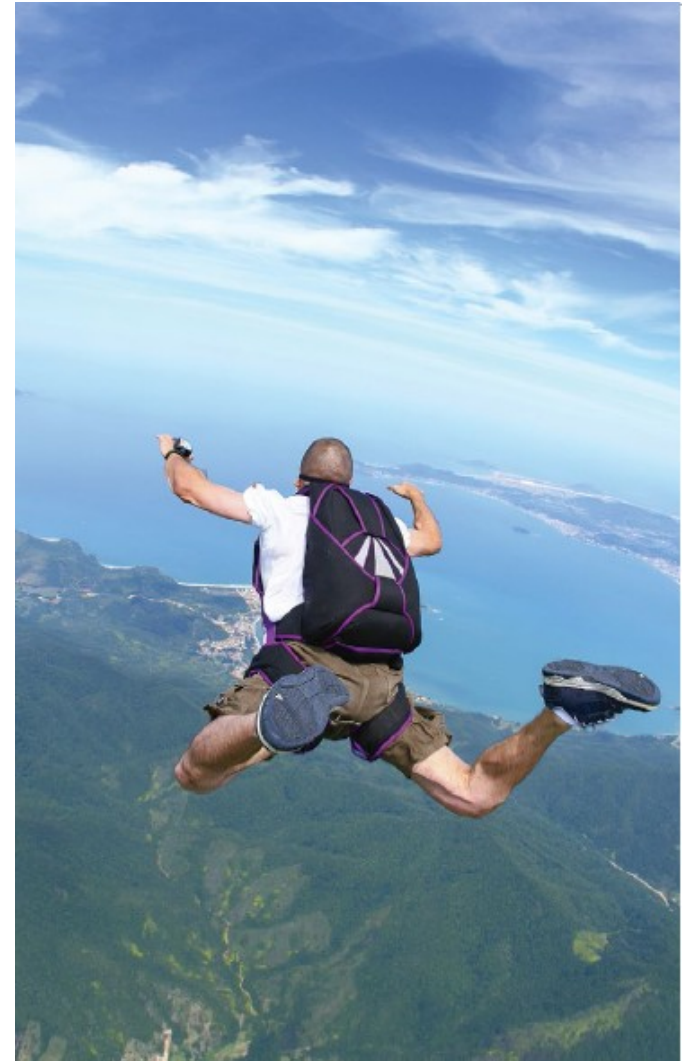
Secondo principio della dinamica:

nella direzione verticale l'**accelerazione** a del paracadutista è determinata dalla legge:

$$ma = mg - F_{att}$$

m è la massa del paracadutista, $g = 9.81 \text{ m/s}^2$ è l'accelerazione di gravità e F_{att} è la **forza di attrito** dovuta alla resistenza dell'aria.

All'inizio la velocità verticale v aumenta, ma dopo un certo tempo il paracadutista raggiunge una **velocità-limite** v_{lim} : adesso $F_{att} = mg$.



7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un'applicazione fisica: **simulazione della caduta libera di un paracadutista.**

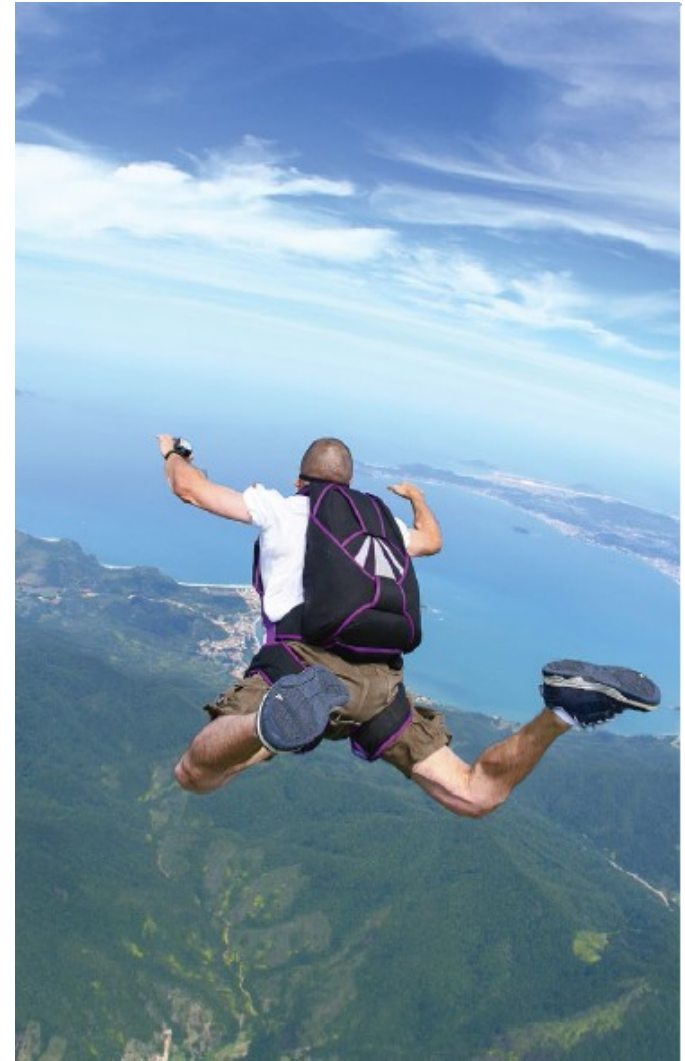
Esistono **due diversi modelli teorici** per prevedere l'effetto dell'aria che scorre intorno al paracadutista durante la caduta libera:

modello laminare: $F_{att}(v) = a_L \cdot v$

modello turbolento: $F_{att}(v) = a_T \cdot v^2$

- se il flusso dell'aria è **laminare**, cioè ordinato, l'attrito è **proporzionale alla velocità**;
- se invece il flusso dell'aria è **turbolento**, cioè vorticoso, l'attrito è **proporzionale al quadrato della velocità**.

Quale tra i due modelli descrive meglio la caduta libera del paracadutista?



7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un'applicazione fisica: **simulazione della caduta libera di un paracadutista.**

L'accelerazione è la derivata della velocità rispetto al tempo, perciò la legge di Newton per il moto del paracadutista si può scrivere come **equazione differenziale del primo ordine**:

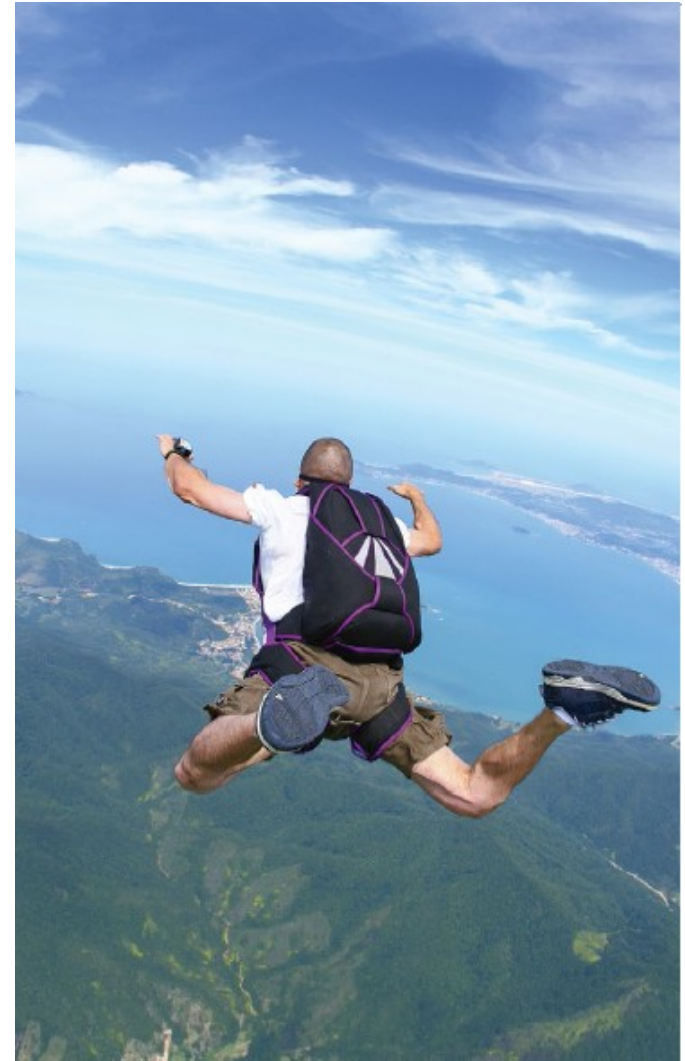
$$v'(t) = g - F_{att}(v(t))/m$$

Imponendo la condizione iniziale $v(t_0 = 0) = 0$, questo diventa un **problema di Cauchy**.

Possiamo allora **simulare numericamente** la caduta, risolvendo con il **metodo di Eulero** le equazioni differenziali nei due casi:

modello laminare: $v'(t) = g - a_L \cdot v/m$

modello turbolento: $v'(t) = g - a_T \cdot v^2/m$



7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un'applicazione fisica: **simulazione della caduta libera di un paracadutista.**

```
eulero_paracad.m x
1 function [vL,vT] = eulero_paracad(t0,T,v0,N)
2 %-----
3 % Risolve l'ODE v'(t)=funz(t) con il metodo di Eulero esplicito.
4 %-----
5 h = (T-t0)/N;           % passo di discretizzazione
6 t = t0:h:T;             % vettore dei tempi
7 vL(1) = v0;
8 vT(1) = v0;
9 m = 80; g = 9.81; aL = 12.8; aT = 0.21;
10 for n=1:N
11     funzL(n)=g-aL.*vL(n)/m;           % caso laminare
12     vL(n+1) = vL(n) + h*funzL(n);
13     funzT(n)=g-aT.*(vT(n)).^2/m;      % caso turbolento
14     vT(n+1) = vT(n) + h*funzT(n);
15 endfor
16 figure;           % produciamo il grafico delle due soluzioni
17 plot(t, vL,'Linewidth',2, t, vT,'Linewidth',3);
18 legend('laminare','turbolento');
19 title('caduta libera del paracadutista - metodo di Eulero con N=100');
20 xlabel('tempo (s)'); ylabel('velocita' (m/s)); axis ([0,35,0,70,]);
21 endfunction
```

questi coefficienti producono la velocità-limite sperimentale: 220 km/h $\approx$ 61 m/s

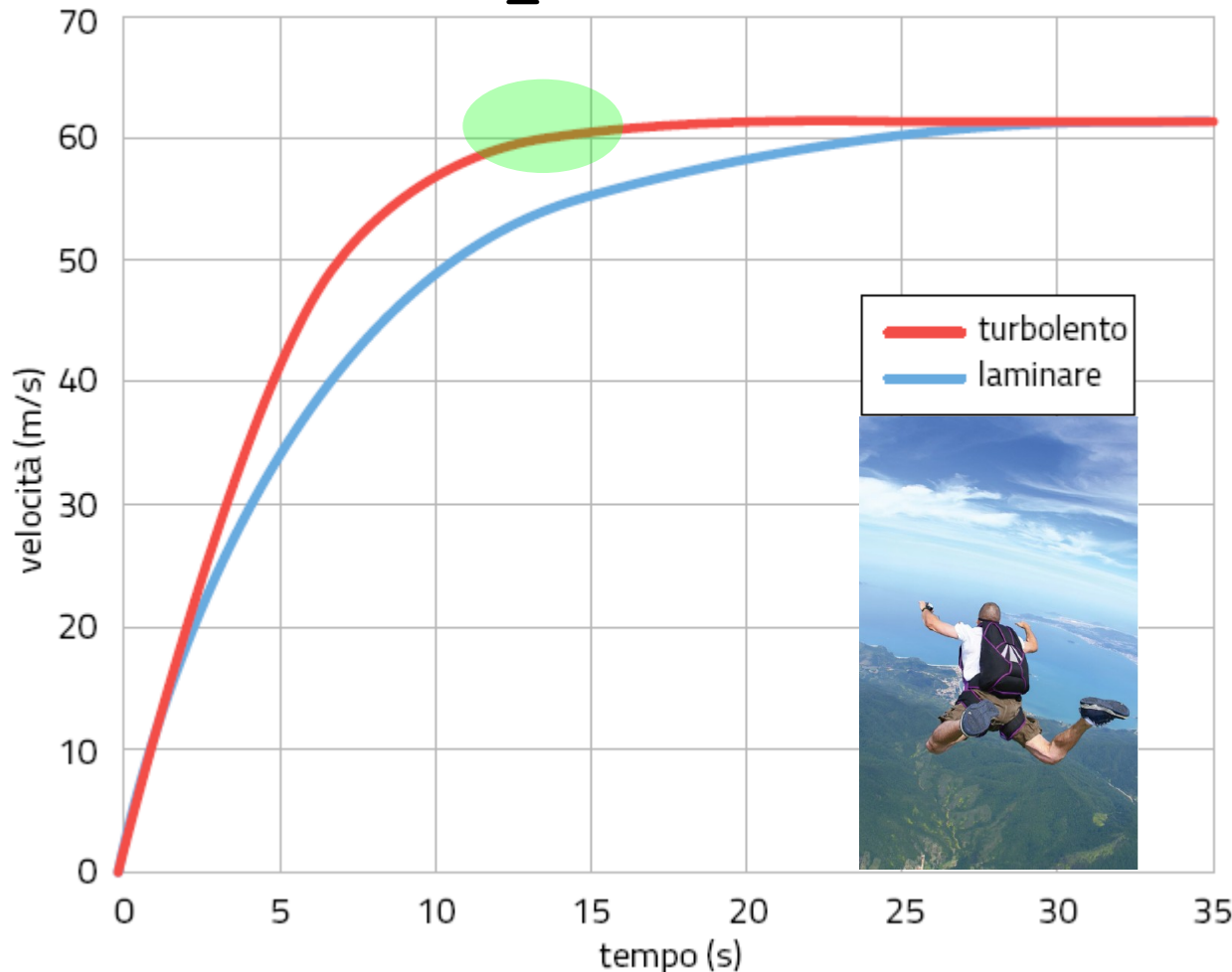
risolve l'equazione differenziale per i due modelli teorici per F_{att}

il grafico confronta le due soluzioni

7.4 Risolvere equazioni differenziali ordinarie: il metodo di Eulero Indice

Un'applicazione fisica: **simulazione** della **caduta libera di un paracadutista**.

`>> eulero_paracad(0, 35, 0, 100)`



esito della simulazione:

con il **modello turbolento** il paracadutista raggiunge la velocità- limite più rapidamente;

con il **modello laminare** invece impiega più tempo.

nella **realtà** la velocità- limite viene raggiunta **in 15 secondi o meno**

conclusione:
il modello corretto
è quello **turbolento**